

嵌入式工作室2026年暑期招新 Robotics方向

第一部分

一、各概念详解

1. 嵌入式系统

定义：嵌入式系统是以应用为中心，以计算机技术为基础，软硬件可裁剪，适用于对功能、可靠性、成本、体积、功耗有严格要求的专用计算机系统。简单说，就是**嵌入到其他设备内部、专门干某一件事情的“小电脑”**。

身边的嵌入式设备：手机、智能手表、路由器、数字电视、微波炉、冰箱、洗衣机、汽车导航、ATM机、无人机、扫地机器人等。

核心特点：专用性（不像PC什么都能干）、软硬件可裁剪、功耗体积受限、可靠性要求高。

2. 单片机 与 STM32

单片机（MCU）：将一个计算机的CPU、存储器、各种I/O接口全部集成到一块芯片上的微型计算机。通俗理解就是一个**芯片上集成了完整的微型电脑系统**。

STM32：是ST（意法半导体）公司基于ARM公司授权的**Cortex-M内核**生产的32位单片机。“STM32”这个名字的含义：

- **ST** = 意法半导体（芯片制造商）
- **M** = Microcontroller（微控制器）
- **32** = 32位处理器

它们的关系：**STM32是单片机的一种**。单片机是大的品类（还包括51单片机、AVR、PIC等），STM32是其中性能较强的一个系列。如果说51单片机是8位的“入门级”，STM32就是32位的“进阶版”，速度更快、资源更多、功能更强。

补充理解：ARM是一家**半导体知识产权公司**，只设计内核不生产芯片；ST是**半导体厂商**，买下ARM的内核授权，再自己加上存储器、外设等，做成完整的STM32芯片。

3. 冯·诺依曼结构 与 哈佛结构

这两种结构描述的是**CPU如何访问指令和数据**。

	冯·诺依曼结构	哈佛结构
指令与数据存储	存在同一存储器中	存在不同的存储器中
总线	共用一条总线	独立的指令总线 and 数据总线
能否同时取指令和取数据	不能（要排队）	能（并行）
速度	较慢	较快

	冯·诺依曼结构	哈佛结构
代表	X86架构、ARM7	STM32 (Cortex-M3) 、ARM9、大部分DSP

STM32内核采用的是哈佛结构。Cortex-M3内核拥有独立的指令总线 and 数据总线，可以让取指与数据访问并行进行，提升效率。

4. 各术语详解与关系梳理

下面这16个术语可以分为三类来理解：

第一类：MCU与外设（整体与部分的关系）

- MCU (Microcontroller Unit, 微控制器) ：就是单片机。它是**核心**，包含了CPU、存储器、和各种**外设**。
- 外设 (Peripheral) ：集成在MCU芯片内部的功能模块，如GPIO、定时器、ADC、串口等。

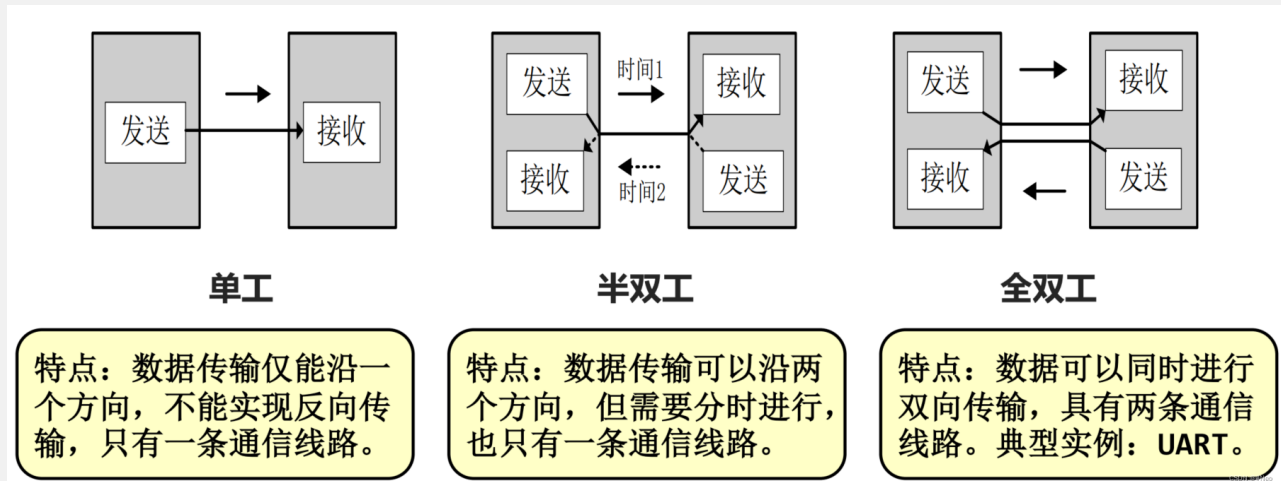
关系：MCU = CPU + 存储器 + 各种外设。外设是MCU的“器官”，MCU通过外设与外部世界交互。

第二类：通信方式（并行/串行、同步/异步）

- 并行通信**：数据的多个位**同时**在多条线上传输（比如8位数据同时用8根线传）。速度快，但占用的引脚多、线路长度受限。
- 串行通信**：数据**一位一位地**在单条线上顺序传输。速度慢一些，但引脚少、成本低、适合长距离。
- 同步通信**：通信双方**共用同一个时钟信号**，数据的收发由这个时钟来同步。效率高，但对时钟精度要求高。
- 异步通信**：通信双方**各自使用自己的时钟**，依靠数据帧中的起始位、停止位来约定节奏（如UART）。对时钟精度要求低，但每帧有额外的开销。

关系：串行通信可以是同步的（如SPI、I2C），也可以是异步的（如UART）。并行通信通常也是同步的。

通信补充：传输模式：单工、半双工（数据传输可以沿两个方向，但是需要分时间进行、只有一条路径）、全双工（两条通信线路）



IIC的通信协议的三根信号线：GND+SCL+SDA

第三类：具体外设与接口

GPIO (通用输入输出端口)

STM32与外界交互的最基本接口，引脚可以配置为**输入**（读取外部信号）或**输出**（对外输出高低电平）。是你点灯、读按键的基础。

串口 / UART / USART

- **串口**是统称，指串行通信接口。
- **UART**（通用异步收发器）：只支持**异步**串行通信。
- **USART**（通用同步异步收发器）：同时支持**同步和异步**串行通信，比UART更强大。

关系：USART是UART的升级版。STM32上通常用USART，但实际开发中大多只使用它的异步模式（当成UART用）。

DMA (直接存储器访问)

可以让数据**不经过CPU**，直接在外设和内存之间搬运数据。比如你要把串口收到的一大串数据存到内存里，用DMA的话CPU可以去做别的事，DMA自己就把活干完了。**核心价值：解放CPU。**

ADC (模数转换器)

将**模拟信号**（连续变化的电压）转换为**数字信号**（CPU能处理的数值）。比如读取光敏传感器、电位器的电压值。

DAC (数模转换器)

将**数字信号**转换为**模拟信号**（输出电压）。是ADC的逆过程，比如用来输出音频信号。

TIM (定时器)

STM32里的定时器功能非常强大，可以：计时、测量频率、**产生PWM波形**、配置编码器接口等。是你做呼吸灯、控制舵机的核心外设。

RTC (实时时钟)

在STM32内部完成**时分秒**的计时功能，可外接备用电池实现**掉电后继续走时**。

SDIO

SD卡接口，用于**读写SD卡**。

USB

生活中最常见的通信协议，STM32上通常作为**从设备**与电脑通信。

CAN (控制器局域网)

主要用于**汽车领域**的通信协议，抗干扰能力强。

SPI（串行外设接口）

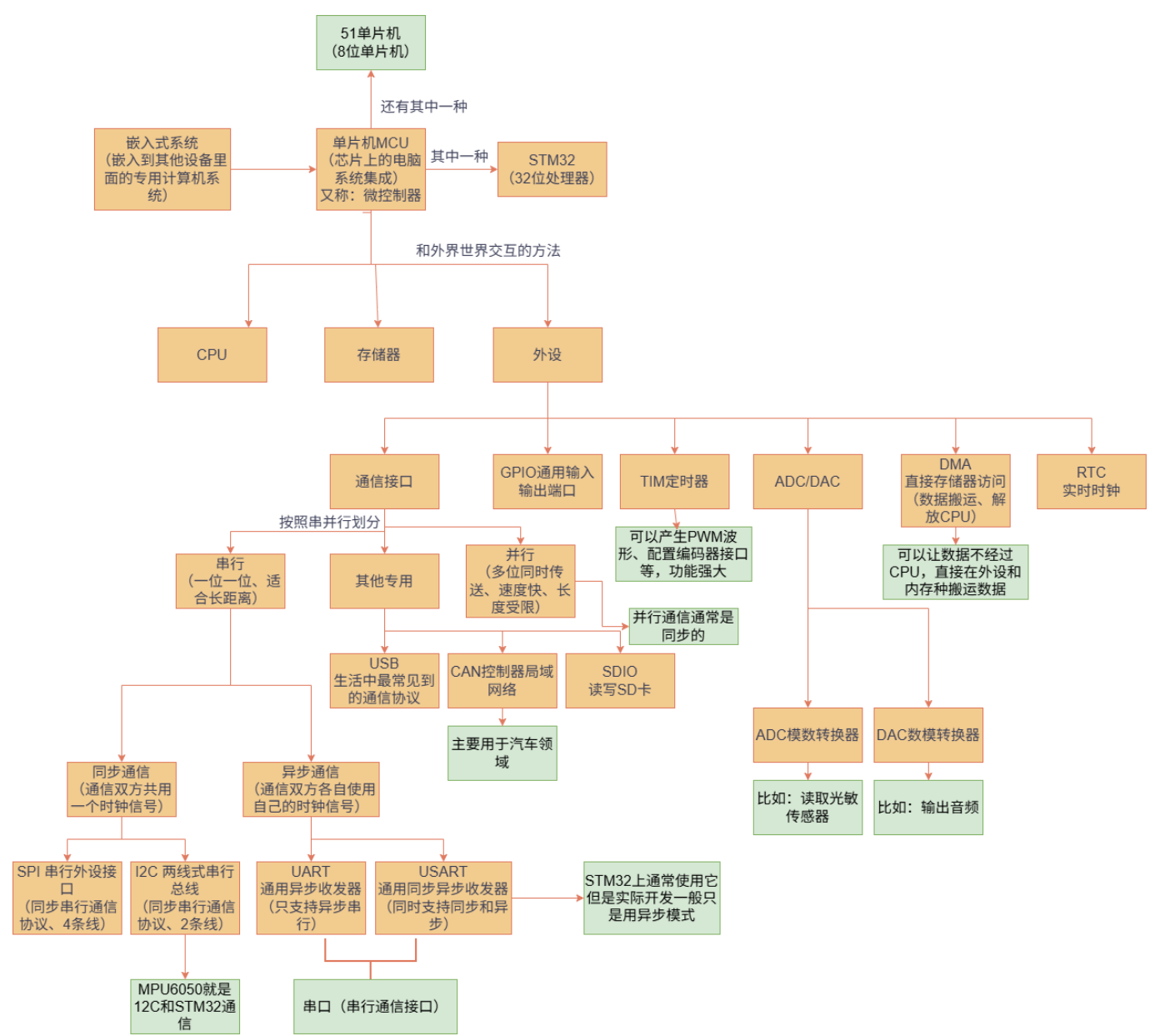
同步串行通信协议，使用4条线（时钟、数据出、数据入、片选），速度较快，适合连接高速设备。

I2C/IIC（两线式串行总线）

同步串行通信协议，只用2条线（时钟SCL、数据SDA），支持**多设备挂载**在同一总线上。MPU6050就是通过I2C与STM32通信的。

起始位：当SCL是高电平时，SDA从高到低的跳变 停止位：当SCL是高电平时，SDA从低到高的跳变

三、概念关系



第二部分

项目包1：

1 使用库函数点亮 STM32F103C8T6 最小系统板上的板载LED灯。 用按键实现LED的可控闪烁。

3 使用delay函数完成led以特定频率闪烁。

2 用GPIO实现流水灯效果。

4 使用光传感器实现“天亮灯灭，天暗灯亮”的功能。

项目包2:

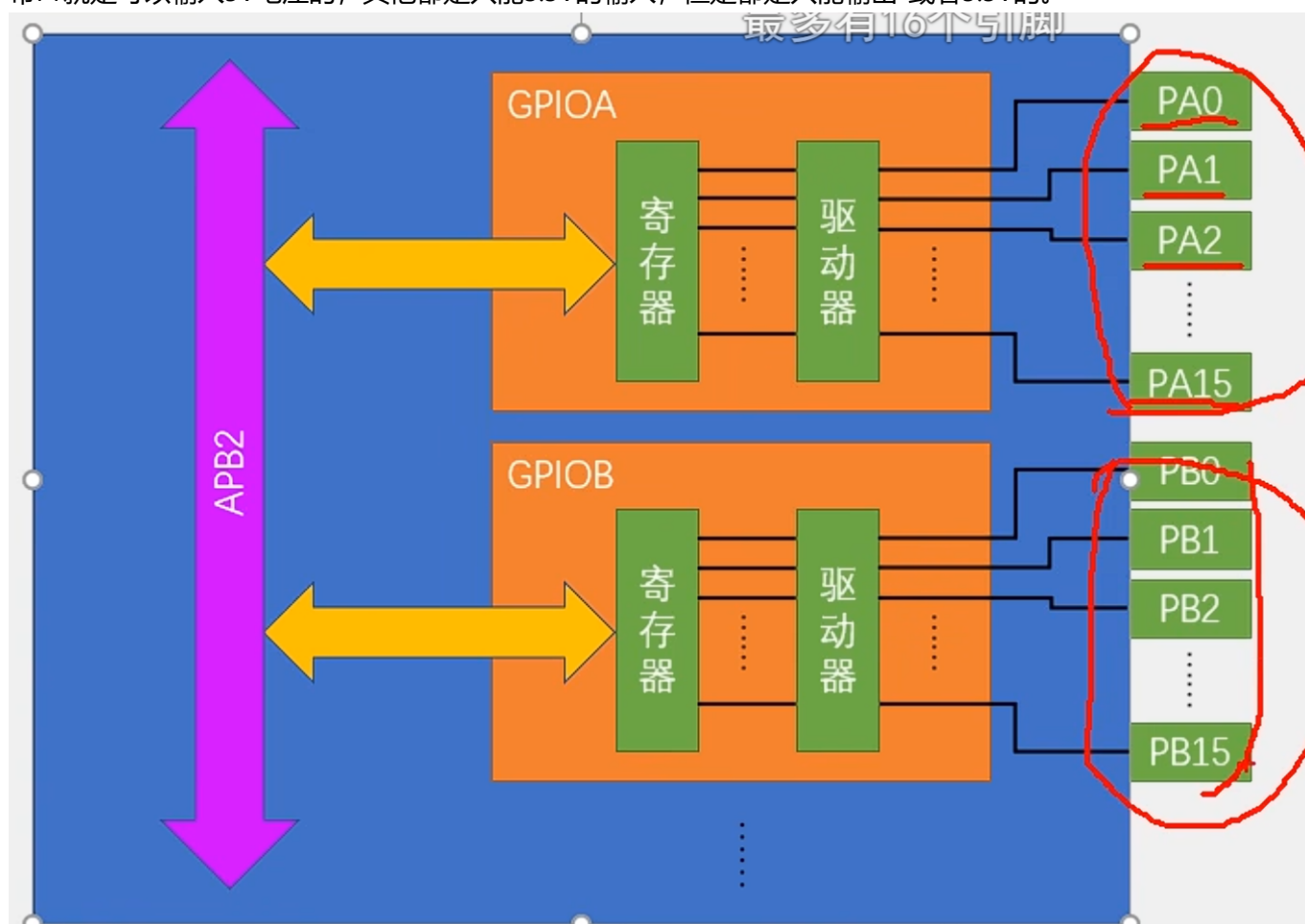
使用PWM完成呼吸灯效果

重点知识补充:

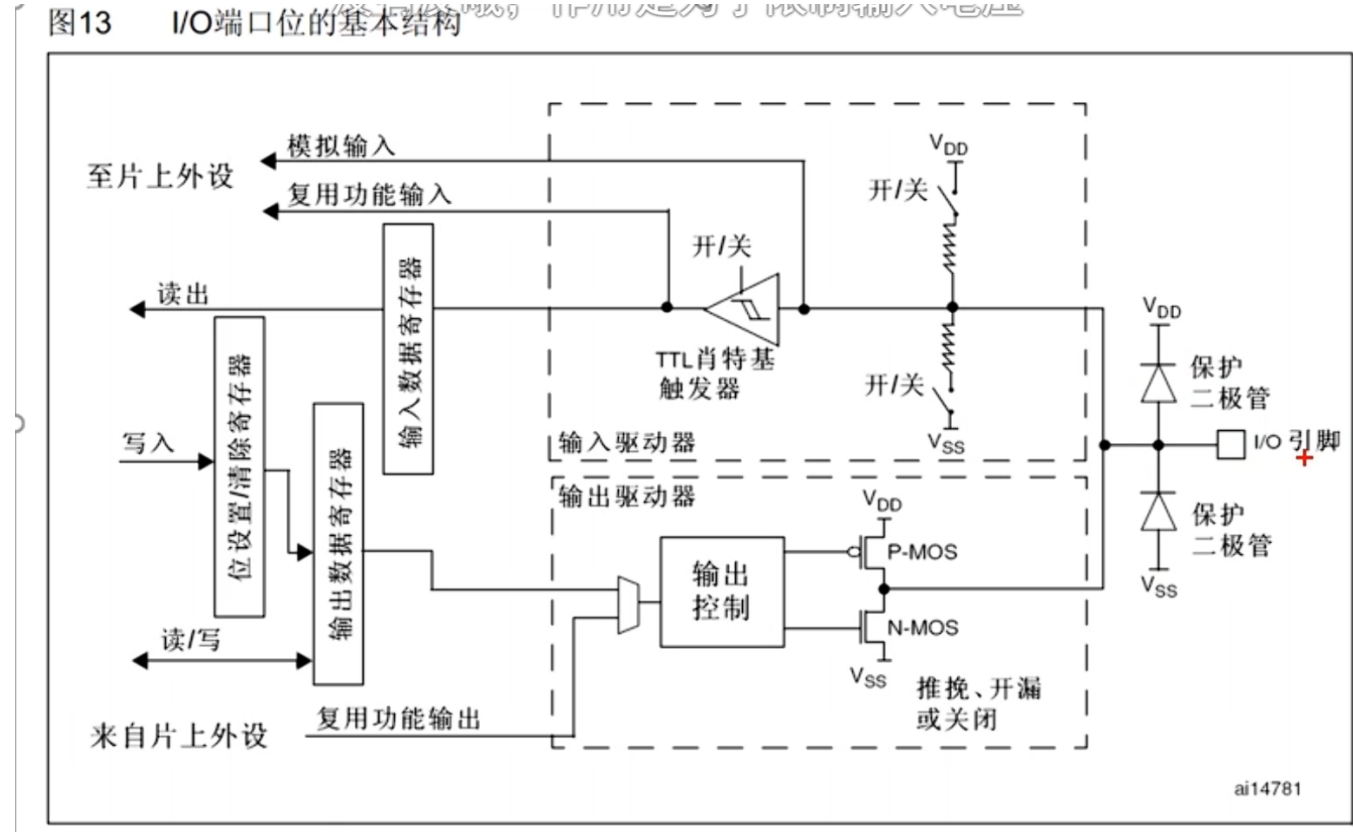
1.GPIO

(1) 可配置为8种输入输出模式:

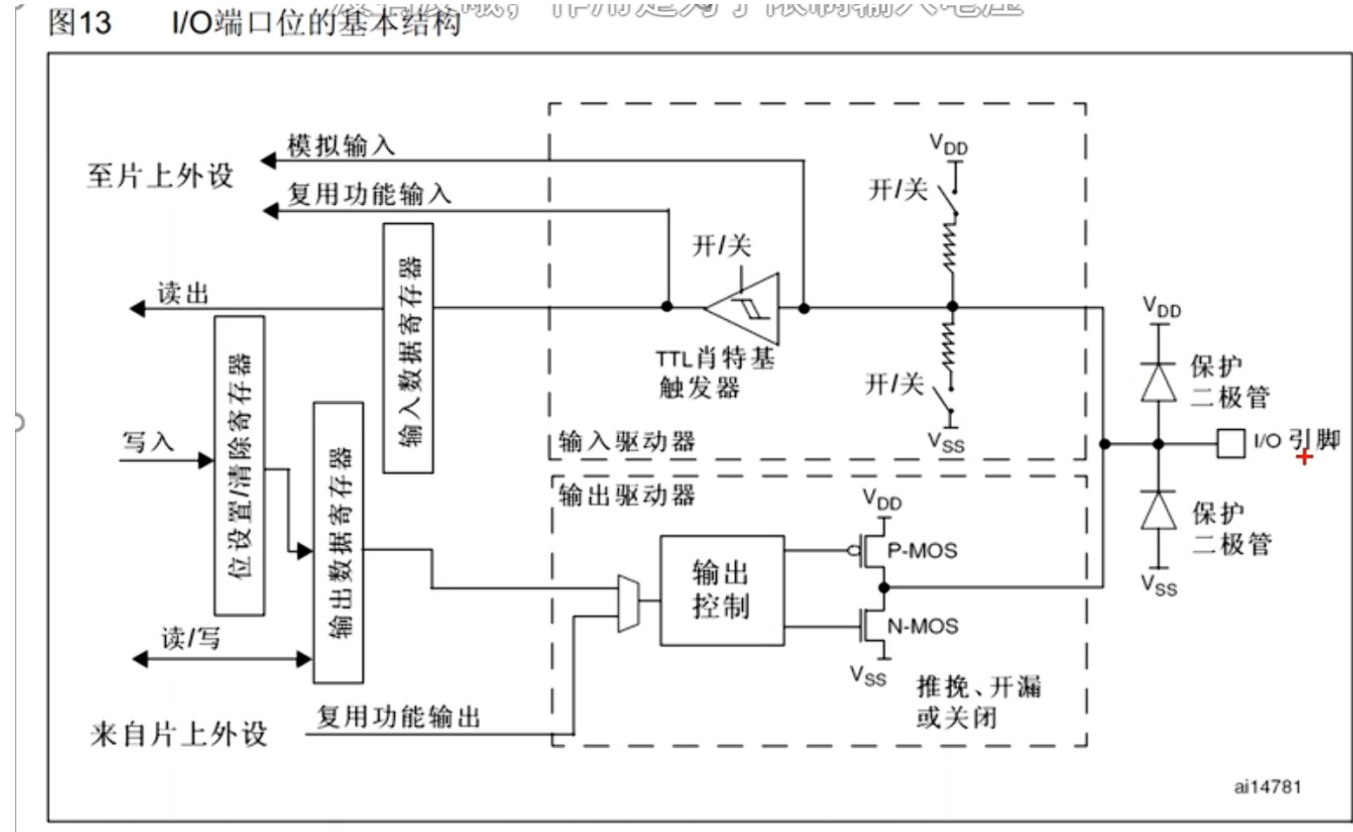
带FT就是可以输入5V电压的，其他都是只能3.3V的输入，但是都是只能输出-或者3.3V的。



STM32是32位的寄存器，所有有32位，但是端口有16位，所以寄存器只有低16位对应有端口



- (2) 具体:
- 1) GPIO_Mode_AIN 模拟输入; 2) GPIO_Mode_IN_FLOATING 浮空输入; 3) GPIO_Mode_IPD 下拉输入;
 - 4) GPIO_Mode_IPU 上拉输入; 5) GPIO_Mode_Out_OD 开漏输出; 6) GPIO_Mode_Out_PP 推挽输出;
 - 7) GPIO_Mode_AF_OD 复用开漏输出; 8) GPIO_Mode_AF_PP 复用推挽输出。



【器件解释】

1、保护二极管

1IO引脚上下两边两个二极管用于**防止引脚外部过高、过低的电压输入**。

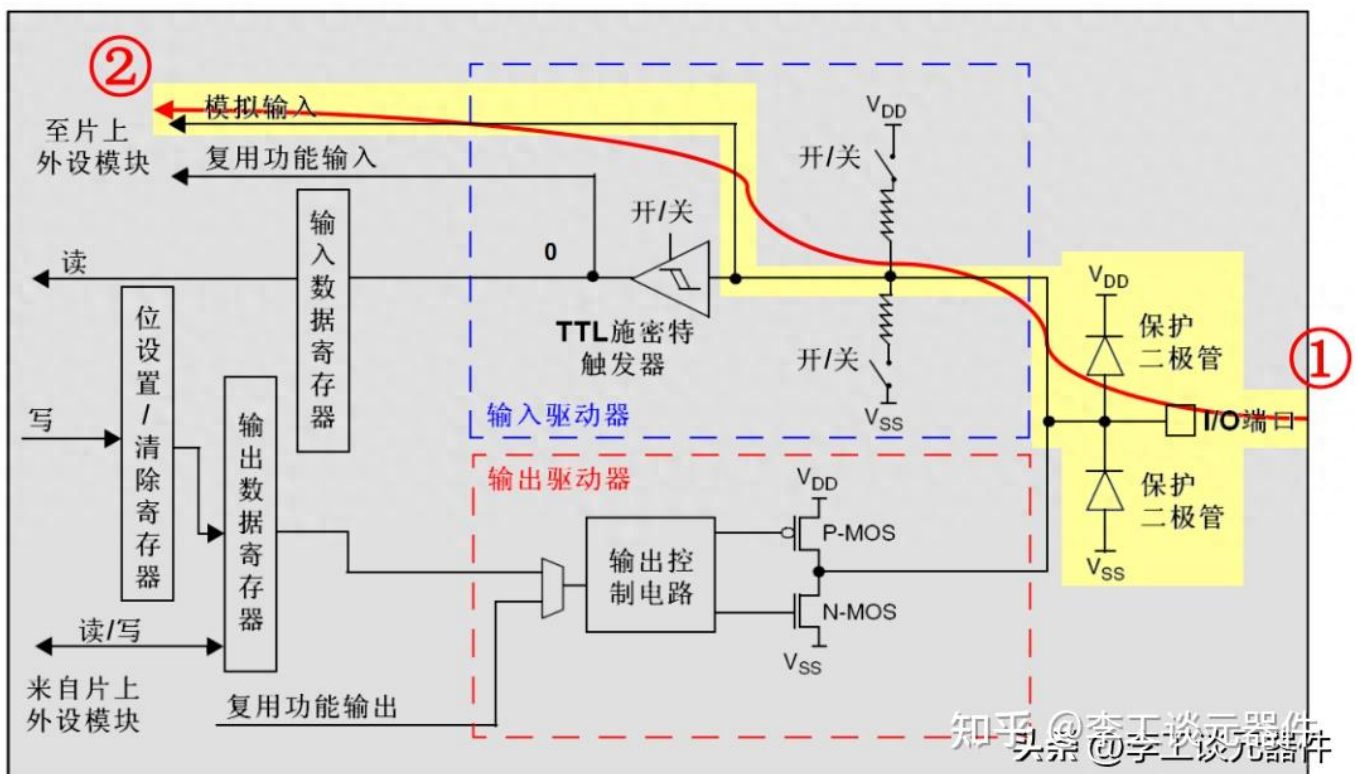
2.TTL 肖特基触发器

信号经过触发器后，模拟信号转化为 0 和 1 的数字信号。但是，当 GPIO 引脚作为 ADC 采集电压的输入通道时，用其**模拟输入功能**，此时信号不再经过触发器进行TTL电平转换。（可以理解为一个比较器，大于某个值是 1，小于某个值是0）

查看《STM32中文参考手册V10》中的GPIO的表格时，会看到有“FT”一列，这代表着这个GPIO口时兼容3.3V和5V的；如果没有标注“FT”，就代表着不兼容5V。

(3) STM 32 GPIO 八种工作模式

1、模拟输入 GPIO_Mode_AIN 此模式可以检测外部输入的模拟电压，可以检测电压值，只要不高于Vcc即可。

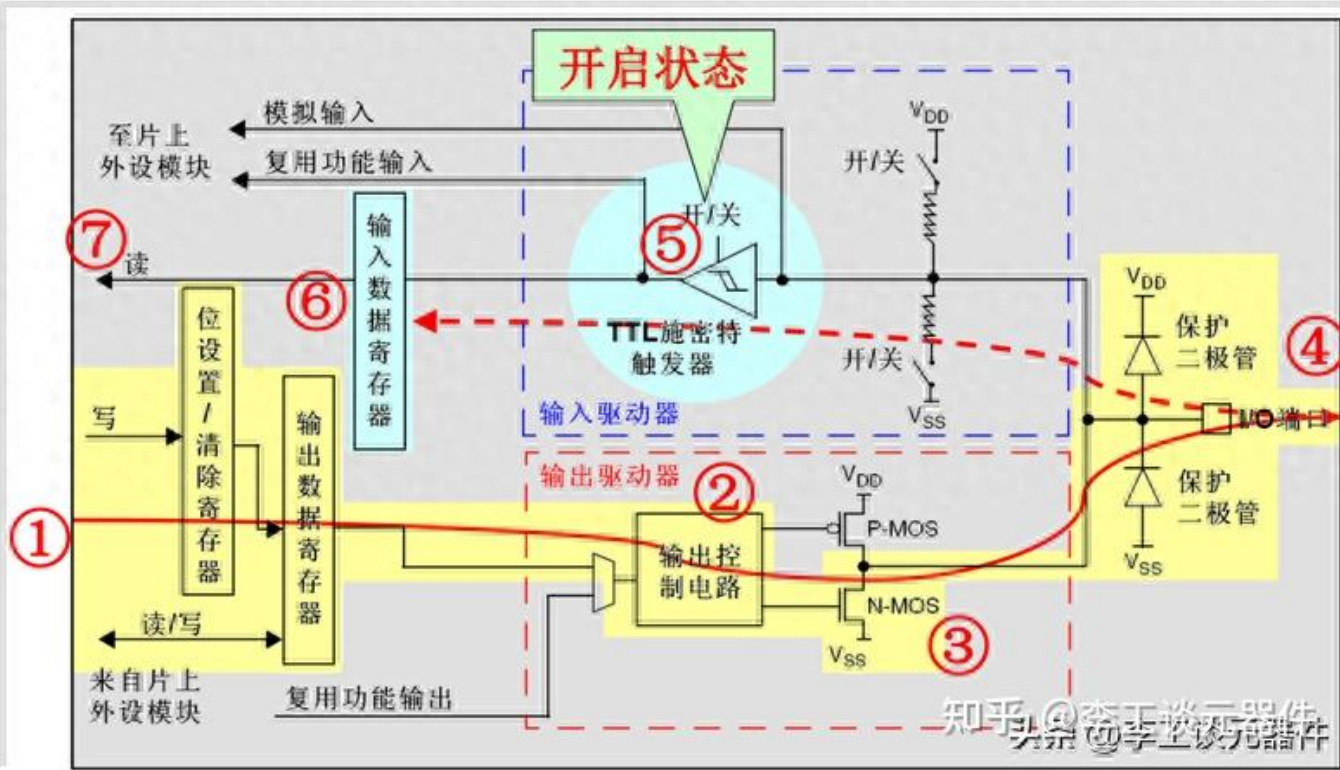


2、浮空输入 GPIO_MODE_IN_FLOATIN 此模式最常用的是检测按键，可以接收高低电平。但容易被干扰。

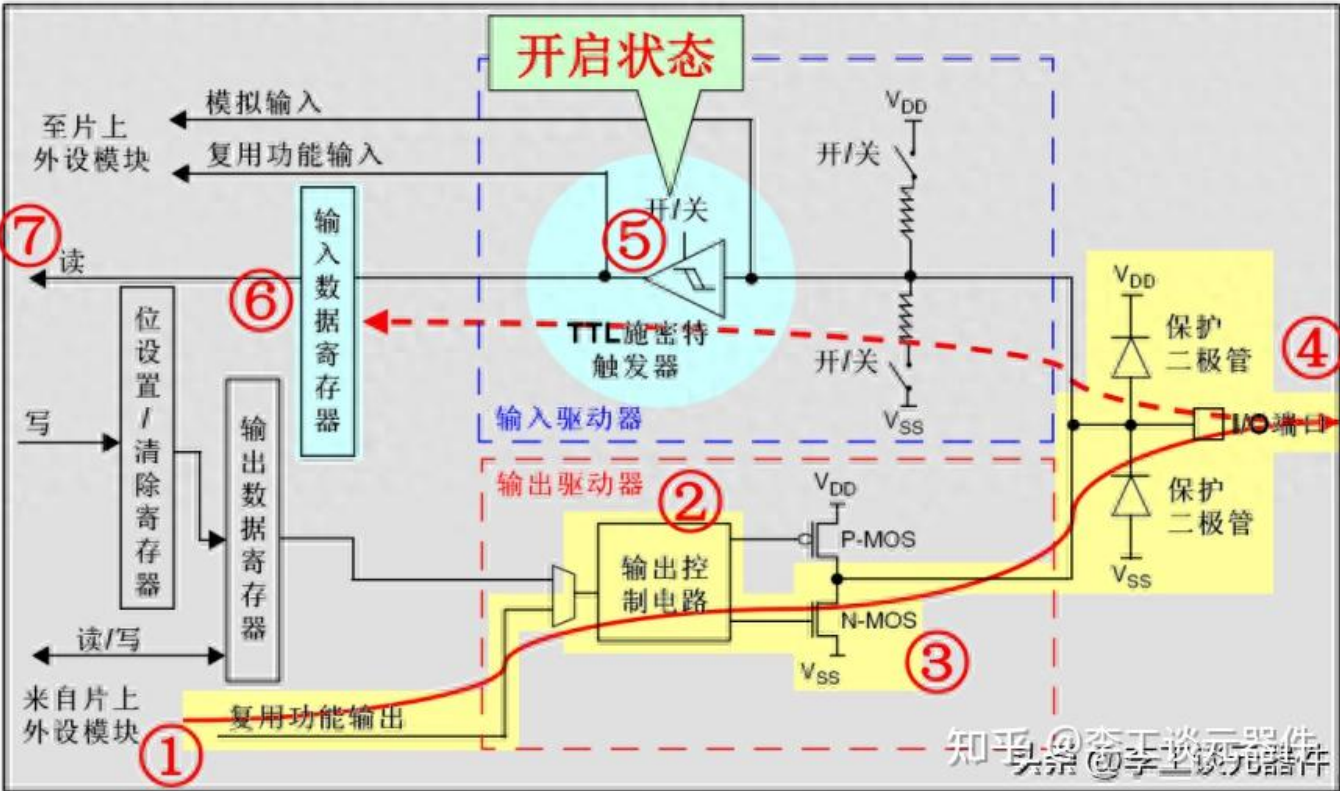
3、下拉输入GPIO_Mode_IPD 此模式检测到电平默认为低，可以检测到由低到高的电平变化。

4、上拉输入GPIO_Mode_IPU 此模式检测到电平默认为高，可以检测到由高到低的电平变化。

5、开漏输出GPIO_Mode_Out_OD 开漏输出用于输出低电平，高电平靠外部上拉电阻电压决定，适用于快速切换电压的外部电路结构。



- 6、推挽输出GPIO_Mode_Out_PP 推挽输出用于输出高低电平，是最常用的模式。
- 7、复用开漏输出GPIO_Mode_AF_OD 复用 IIC 时候选择复用开漏输出，因为开漏输出可以“线与”。



- 8、复用推挽输出GPIO_Mode_AF_PP 其他复用比如 SPI 等可以选择复用推挽输出。

区别开漏输出和推挽输出这两个常用的：

****开漏输出的速度比推挽输出慢得多****，它真正厉害的地方是 *****“电平转换”***** 和 *****“多设备共线”*****

(线与) ”**。

1. 推挽输出 (GPIO_Mode_Out_PP) —强硬!!!

- ****内部结构****: 芯片内部有一个****P-MOS管 (负责接高电平VDD) **** 和一个****N-MOS管 (负责接低电平VSS) ****。
- ****工作逻辑****:
 - 输出`1` (高电平): P-MOS导通, N-MOS关闭, ****直接短路到3.3V电源****, 强力输出高电平。
 - 输出`0` (低电平): P-MOS关闭, N-MOS导通, ****直接短路到地 (GND) ****, 强力输出低电平。
- ****特点****: 高低电平都有强驱动能力, ****上升沿和下降沿都非常陡峭****, 速度快。适合点灯、驱动舵机 (PWM)、驱动蜂鸣器。
- ****死穴****: ****不允许两个推挽输出引脚直接连在一起! **** 如果A输出高(3.3V), B输出低(0V), 两个P-MOS和N-MOS同时导通, 相当于****电源直接对地短路****, 瞬间大电流会把芯片烧掉。

2. 开漏输出 (GPIO_Mode_Out_OD)

- ****内部结构****: 芯片内部****只有1个N-MOS管****, ****没有P-MOS管****。上面是断开的 (高阻态)。
- ****工作逻辑****:
 - 输出`0` (低电平): N-MOS导通, ****强力拉到GND****。
 - 输出`1` (高电平): N-MOS****完全关闭****, 引脚处于****高阻态 (Hi-Z) ****, 内部相当于把引脚“断开”了。 ****芯片自己绝对不输出高电平! ****
- ****高电平怎么来****: ****必须在外加一个上拉电阻 (比如4.7KΩ或10KΩ) ****, 电阻的另一头接想用的电压 (可以是3.3V, 也可以是5V)。N-MOS关闭时, 引脚就被电阻拉到了这个外部电压。

开漏输出依赖外部上拉电阻, 而上升沿的电压是通过****电阻给寄生电容充电****建立的。电阻越大, 上升越慢 (几微秒); 电阻越小, 上升快但功耗大。

- ****推挽速度****: 纳秒级 (极快)。
- ****开漏速度****: 微秒级 (较慢)。所以它****绝对不适合高速通信**** (比如SPI的高速模式)。I2C标准模式只用400kHz, 就是因为开漏跑不快。

3. 既然开漏慢又麻烦, 为什么还要用它?

****原因一: 用来匹配不同电压的外部设备 (电平转换) ****

你的STM32是3.3V逻辑, 但有些老式模块 (或OLED) 可能需要5V电平。开漏模式下, ****STM32内部不输出高电平****, 高电平完全由外部上拉电阻决定。

- 如果你想把3.3V单片机接到5V的I2C设备, 只需在SDA和SCL上加****上拉电阻到5V****。STM32内部输出低电平时拉低到0V, 输出高电平时“放手”让5V电阻拉上去。 ****完美实现电压匹配, 还不伤芯片。 ****

****原因二: 支持“线与”总线 (多设备不打架) —这是I2C (MPU6050) 必须用开漏的根本原因! ****

I2C总线上挂了很多设备 (MPU6050、OLED等)。如果主机和从机都用推挽输出:

- 主机想发1 (拉高), 从机想发0 (拉低), 瞬间电源短路, ****芯片必烧****。

用了开漏模式:

- 主机发1时: N-MOS关闭 (放手)。
- 从机发0时: N-MOS导通 (拉低)。
- ****结果****: 因为主机关闭了, 从机能轻松把总线拉低; 反过来也一样。没有任何短路电流。

****结论****: I2C协议****强制要求****信号线 (SCL、SDA) 必须配置为****开漏输出****, 配合外部上拉电阻。这就是为什么之后做MPU6050时, 一定要在PB6/PB7上接两个4.7K电阻, 并且在初始化GPIO时用`GPIO\_Mode\_AF\_OD` (复用开漏输出)。

```
### 4. 针对这一项目的选择
| 场景 | 用哪个模式 | 为什么 |
| :--- | :--- | :--- |
| **点亮LED / 驱动舵机PWM** | **推挽输出** | 需要大电流驱动，速度要快，电平稳定。 |
| **按键输入检测** | 输入模式（上拉/下拉） | 输出模式是用于控制外部电路，读按键要配输入模式，别配错了。 |
| **MPU6050 的 I2C 通信** | **复用开漏输出** | I2C协议强制要求，必须配合外部上拉电阻，防止总线冲突烧芯片。 |
| **蓝牙 HC05 的 TX/RX** | **复用推挽输出 (TX) / 浮空输入 (RX)** | 串口是点对点通信（一对一），只有一个发一个收，不存在总线冲突，用推挽跑得快。 |

开漏输出用于输出低电平，高电平靠外部上拉电阻电压决定，适用于需要进行电平转换（匹配 3.3V/5V）以及需要多设备共线（如I2C总线）的场景。
```

项目包1-2的重要代码解释部分【附件1】

按键函数

PWM呼吸灯函数

光感受器函数

附加题

项目包3：使用OLED显示你的学号、姓名，并且使用OLED显示一张图片

学习笔记：

1.OLED

1) 工作原理：OLED是一种利用有机材料发光的显示技术

(1) 两个电极：一个是透明的阳极（通常用ITO材料），另一个是金属阴极。（2）有机发光层：夹在两个电极之间，厚度仅有几十纳米。

当给它通电时：阴极会注入电子（带负电），阳极会注入空穴（可以理解为带正电的“空洞”）。在电场作用下，电子和空穴会相向移动，在中间的有机发光层里相遇并结合。它们结合时释放的能量会激发有机分子，分子从激动状态回到稳定状态时，就会以光的形式释放出多余的能量。光的颜色由有机发光材料决定，而亮度则由电流大小控制。

(3) 特点：OLED是自发光的

2) OLED显示中文

(1) 背景：内部只内置了英文字母和数字的字模（ASCII字符集），必须为要显示的中文提供“字模”（即每一个汉字在OLED上的点阵图案数据）

(2) 方法：详见项目包3

第三部分：项目一——舵机云台

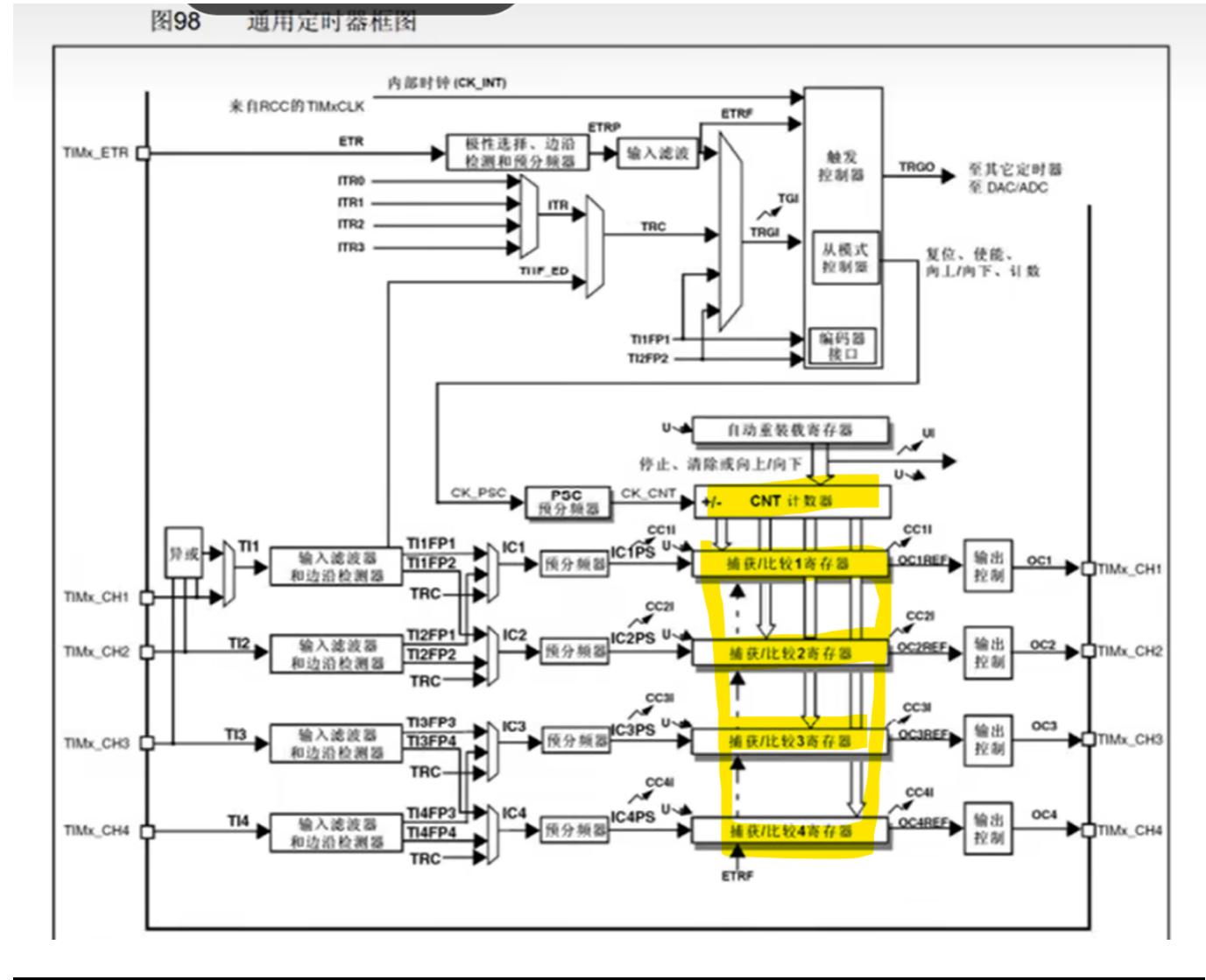
挑战点一：使用PWM驱动舵机

项目包4

学习笔记

1.TIM的输出比较——有电机的项目重点，比如：PWM驱动电机

英文缩写（常见的）：OC



2.PWM——脉冲宽度调制

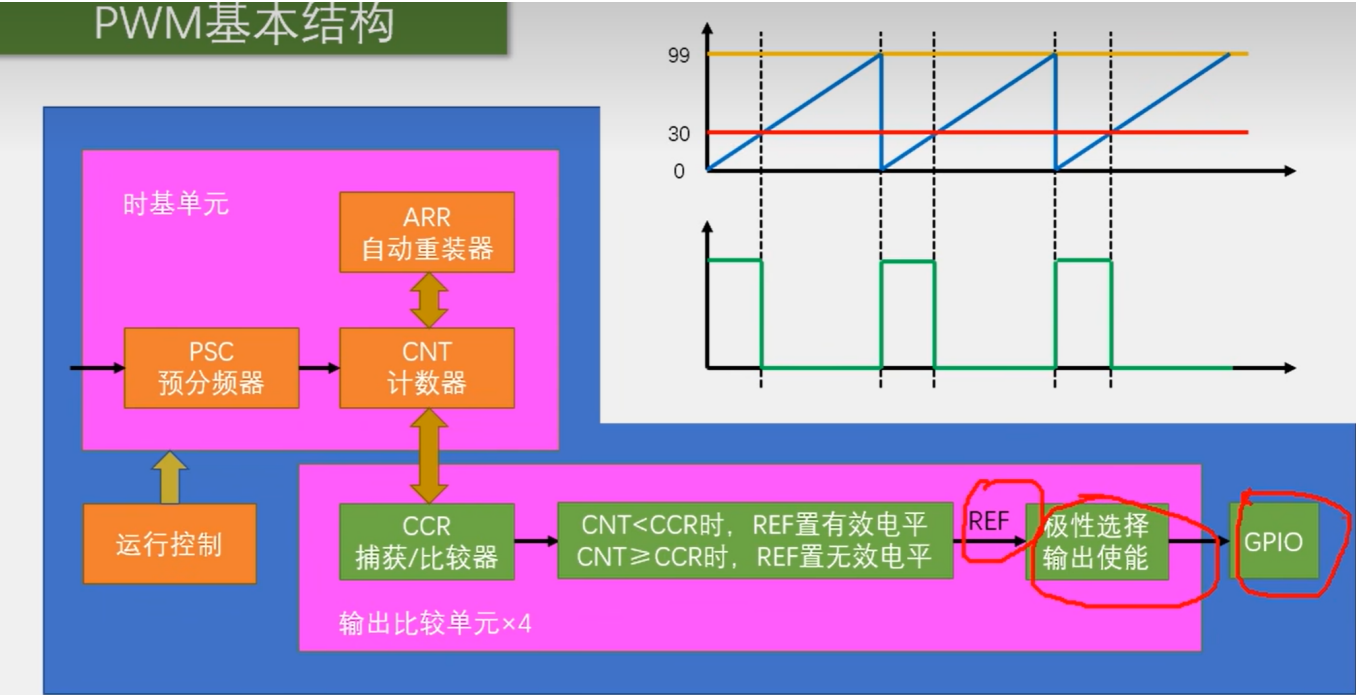
简而言之使用PWM呼吸灯来讲，就是快速多次的进行高低电平的交替，使得呈现一种中等亮度（一种等效的模拟量）

要求：具有惯性的系统（如：呼吸灯、电机）

参数：频率、占空比（高电平占一个周期内的时间长度）、分辨率（占空比变化的细腻程度）

模式	描述
冻结	CNT=CCR时，REF保持为原状态
匹配时置有效电平	CNT=CCR时，REF置有效电平
匹配时置无效电平	CNT=CCR时，REF置无效电平
匹配时电平翻转	CNT=CCR时，REF电平翻转
强制为无效电平	CNT与CCR无效，REF强制为无效电平
强制为有效电平	CNT与CCR无效，REF强制为有效电平
PWM模式1	向上计数：CNT<CCR时，REF置有效电平，CNT≥CCR时，REF置无效电平 向下计数：CNT>CCR时，REF置无效电平，CNT≤CCR时，REF置有效电平
PWM模式2	向上计数：CNT<CCR时，REF置无效电平，CNT≥CCR时，REF置有效电平 向下计数：CNT>CCR时，REF置有效电平，CNT≤CCR时，REF置无效电平

2-4可以用于形成PWM的波形，2-3是输出一次性的信号，1和5-6都是用于暂停



这张图是一个STM32定时器PWM生成的核心原理框图，它把“时基”、“比较”和“输出路径”三个环节串在了一起。结合你之前的舵机代码，我来帮你把这张图彻底“翻译”成人话：

图片知识解释：

- 纵坐标代表 **计数器 (CNT)** 的数值。
- 假设 **ARR (自动重装器) = 99**，那么 CNT 就会从 **0** 一直往上数到 **99**，然后溢出归零，重新再数（即 **0 → 30 → 99 → 0** 的三角波/锯齿波）。
- 图中的 **30** 代表写入 **CCR (捕获/比较器)** 的当前值。

这个 REF 信号就是一个**占空比 = 30 / (99+1) ≈ 30%** 的原始方波。**CCR 的值决定了高电平持续多久（占空比）**，这就是 PWM 的核心。

图里横坐标指向了 **REF** → **极性选择** → **输出使能** → **GPIO**，这代表原始信号 **REF** 要经过“三关”才能变成引脚上的实际电压：

1. **REF (原始信号)**
2. **极性选择**：你可以选择“不反转”（高电平有效）或“反转”（低电平有效）。你代码里写的 `TIM_OCPolarity_High` 就是告诉硬件“别反转，直接过”。
3. **输出使能**：相当于一个总开关，只有开启（`TIM_OutputState_Enable`），信号才能往外送。
4. **GPIO**：最终信号到达物理引脚（如 PA1），变成示波器上看到的实际波形。

一句话总结：PSC 和 ARR 决定方波的“周期”（多宽），CCR 决定方波的“宽度”（高电平多长），REF 经过极性和使能后从 GPIO 输出。

【重点！！】 这三个公式并非凭空定义，而是由定时器的 **硬件计数原理** 直接推导出来的。简要推导如下：

1. 频率公式的来源

$$[F_{\text{PWM}} = \frac{CK_PSC}{(PSC + 1) \times (ARR + 1)}]$$

- **计数脉冲**：定时器时钟（CK_PSC）先经过预分频器（PSC）分频，得到实际驱动计数器的时钟频率：

$$[f_{\text{CNT}} = \frac{CK_PSC}{PSC + 1}]$$
- **一个周期**：计数器从（0）向上计数到（ARR）需要计（ARR + 1）个数，每个数耗时（1/f_CNT）

2. 占空比公式

$$[\text{Duty} = \frac{CCR}{ARR + 1} \times 100\%]$$

- 计数器从（0）跑到（ARR）（共（ARR+1）个计数值）。
- 输出电平在（CNT < CCR）时有效，有效时长对应（CCR）个计数值。

3. 分辨率公式的来源

$$[\text{Resolution} = \frac{1}{ARR + 1}]$$

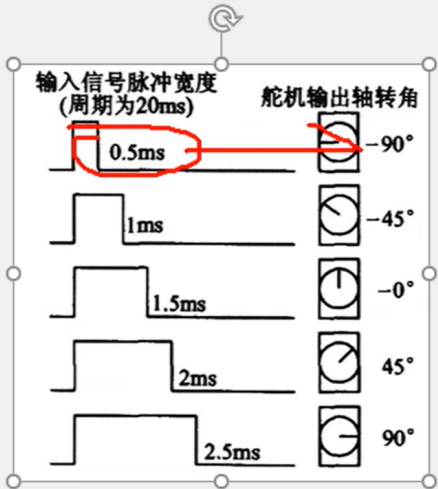
- （CCR）寄存器只能按 **整数 1** 变化，这是硬件限制的最小调节步长。
- 当（CCR）变化 1 时，占空比的变化量为：

$$[\Delta \text{Duty} = \frac{(CCR + 1) - CCR}{ARR + 1} = \frac{1}{ARR + 1}]$$
- 这个最小变化量即为该 PWM 能达到的控制精度（分辨率）。

3. 舵机介绍

舵机简介

- 舵机是一种根据输入PWM信号占空比来控制输出角度的装置
- 输入PWM信号要求：周期为20ms，高电平宽度为0.5ms~2.5ms



挑战一思考题回答

针对你的五个思考题，我按“硬件外设 → 引脚配置 → 执行原理 → 参数计算 → 中断机制”的逻辑线，逐一给出简要且透彻的回答：

1. PWM由什么外设产生？核心参数有哪些及作用？

- **产生外设：**定时器（通用/高级定时器）的输出比较通道产生。本质是利用计数器（CNT）与比较寄存器（CCR）数值匹配时翻转电平。
- **三大核心参数**（由时基+比较器决定）：
 - **频率** (F_{PWM})：由 **PSC** 和 **ARR** 共同决定，决定方波快慢（舵机要求 50Hz）。
 - **占空比** (Duty)：由 **CCR** 决定，决定高电平占比（控制舵机转角）。
 - **分辨率** (Resolution)：由 **ARR** 决定，决定 CCR 每变化 1 时占空比的最小步进（决定转动细腻度）。

2. PWM输出对应的GPIO配置模式？与普通推挽核心区别？

- **配置模式：**复用推挽输出 (`GPIO_Mode_AF_PP`)。
- **核心区别：**
 - **普通推挽输出：**引脚电平由 **CPU 写 ODR 寄存器** 控制，软件翻转速度慢且占用内核资源。
 - **复用推挽输出：**引脚电平由 **片上外设（定时器）直接硬件驱动**，断开 CPU 控制。定时器以纳秒级硬件速度自动翻转电平，且不占用 CPU 算力，这是产生高频稳定 PWM 的根本原因。

需要人为干预、随性变化、或简单开关”（如点灯、按键扫描、软件模拟协议）→ 必须用普通推挽。需要“高频率、纳秒级准时、且靠硬件自动运行”（如舵机 PWM、电机调速、串口通信、SPI 高速时钟）→ 必须用复用推挽。

3. 舵机为何依靠PWM控制转角？高电平时长与角度对应关系？

- **控制原理**：舵机内部有**基准电路**和**电位器反馈**，它并不关心 PWM 的频率占比，而是靠检测 **高电平持续脉宽（μs）** 来决定输出轴转角。低电平期间是舵机内部处理器采样的“空闲期”。
- **对应关系**（标准 50Hz，周期 20ms）：
 - **0.5ms 高电平** → **0°**（极限左）
 - **1.5ms 高电平** → **90°**（中位）
 - **2.5ms 高电平** → **180°**（极限右）

4. 舵机标准 50Hz PWM（20ms），72MHz 时钟，计算 PSC 和 ARR

题目参数	对应寄存器名	符号
72 MHz	定时器时钟	CK_PSC
50 Hz	目标PWM频率	F_PWM
20 ms	PWM周期	T_PWM
待求	预分频器	PSC
待求	自动重装载值	ARR

$$[F_{PWM} = \frac{CK_PSC}{(PSC + 1) \times (ARR + 1)}]$$

第一步：定 ARR（为了分辨率高且 1 计数 = 1 μs）

- 周期 20 ms = 20000 μs，令计 20000 次溢出。
- 所以：**ARR + 1 = 20000 → ARR = 19999**

第二步：代入公式求 PSC $[50 = \frac{72,000,000}{(PSC + 1) \times 20,000}] [(PSC + 1) \times 20,000 = \frac{72,000,000}{50} = 1,440,000] [PSC + 1 = \frac{1,440,000}{20,000} = 72] [PSC = 71]$

寄存器	值
PSC	71
ARR	19999

5. 按键外部中断完整触发流程及全套配置步骤

核心知识点 EXTI（外部中断/事件控制器）和 NVIC（嵌套向量中断控制器）

- **触发流程**：按键电平变化 → EXTI（外部中断线）检测到边沿跳变 → NVIC（嵌套向量中断控制器）根据优先级响应 → CPU 跳转执行中断服务函数（ISR）。
- **全套配置步骤**（共 5 步，请按顺序）：
 1. **配置 GPIO**：开启 GPIO 时钟，将按键引脚设为 **输入模式**（浮空/上拉，取决于按键电路，若按键接 GND 则开启上拉输入）。

2. **配置 AFIO (复用功能IO)**：开启 AFIO 时钟，调用 `GPIO_EXTILineConfig()` 将该 GPIO 引脚映射到对应的 EXTI 线（如 PA0 映射到 EXTI0）。
3. **配置 EXTI**：设置 EXTI 线的**触发边沿**（上升沿/下降沿/双边沿），并使能 EXTI 中断（`EXTI_Init()`）。
4. **配置 NVIC**：设置该中断通道的**抢占优先级和响应优先级**，并使能 NVIC 响应（`NVIC_Init()`）。
5. **编写中断服务函数**：在对应的中断向量入口（如 `EXTI0_IRQHandler`）中，**务必先清除中断标志位**（`EXTI_ClearITPendingBit()`），再执行消抖和按键逻辑（避免重复触发）。

【附件1】项目包1-2

按键函数 PWM呼吸灯函数（附件三详细阐述） 光敏传感器函数

【附件2】项目包3

OLED函数以及中文显示部分

```
#include "stm32f10x.h"
#include "OLED_Font.h"
#include "OLED.h"

/* 引脚配置宏 */
#define OLED_W_SCL(x)      GPIO_WriteBit(GPIOB, GPIO_Pin_8, (BitAction)(x))
#define OLED_W_SDA(x)      GPIO_WriteBit(GPIOB, GPIO_Pin_9, (BitAction)(x))

/* 问题：一般都会使用这两个函数名的修改名字后的宏常量吗？
   答：是的，这种宏定义是标准做法，便于移植和阅读，把硬件操作抽象为函数名。
   补充：宏定义中的 GPIO_WriteBit 函数第三个参数需要强制类型转换 (BitAction)(x)，确保传入0/1。 */

/* 缓冲区：8页 × 128列，每字节代表一列上的8个垂直像素（1 bit=1像素） */
static uint8_t OLED_Buffer[8][128];
/* 问题：缓冲区是什么？
   答：缓冲区是内存中的一块区域，保存了当前屏幕所有像素点数据。操作时先在缓冲区修改，再一次性或按需刷到屏幕，避免频繁I2C通信，提高效率并支持画点等操作。
   补充：缓冲区的内容与屏幕显存一一对应，任何修改屏幕的函数都应同步更新缓冲区，否则画点等操作会失效。 */

static uint8_t current_page = 0;    // 当前页（0~7）
static uint8_t current_col = 0;     // 当前列（0~127）

/* 引脚初始化 */
void OLED_I2C_Init(void)
{
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);
    /* 问题：OLED必须在APB2吗？必须在GPIOB吗？
       答：所有GPIO时钟都在APB2，不是OLED特殊。GPIOB是开发板硬件连接决定的，可以换成其他GPIO，只要修改引脚宏定义。 */

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_OD;
```

/* 问题：哪些外设和OLED是一样的模式？这个模式的原理、选择原因？

答：I2C总线设备（如EEPROM、温湿度传感器）都用开漏输出。原理：输出高电平靠外部上拉电阻，低电平由NMOS拉低。

选择原因：I2C是多主机总线，开漏可实现“线与”功能，避免总线冲突。OLED的I2C接口要求SDA/SCL为开漏。

补充：若使用硬件I2C外设，GPIO模式也必须配置为复用开漏（AF_OD）。这里使用软件模拟I2C，所以普通开漏即可。 */

```
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
```

/* 问题：OLED选择50的原因？修改成其他会有什么变化？

答：I2C速率通常400kHz以下，2MHz都够，但50MHz是通用配置，不影响功能，只是功耗略高。改低不影响功能，可降低EMI。 */

```
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_8;
```

```
GPIO_Init(GPIOB, &GPIO_InitStructure);
```

```
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_9;
```

```
GPIO_Init(GPIOB, &GPIO_InitStructure);
```

```
GPIO_WriteBit(GPIOB, GPIO_Pin_8, Bit_SET);
```

```
GPIO_WriteBit(GPIOB, GPIO_Pin_9, Bit_SET);
```

/*

问题：GPIO_WriteBit的参数：无论是BSRR还是BRR，它们都是“写1有效，写0无效”，所以只需要传入0或1，而不需要读取现在状态？

答：正确。传入Bit_SET (1) 操作BSRR置位，传入Bit_RESET (0) 操作BRR复位，硬件自动处理，无需读当前状态。

补充：初始化时拉高SDA和SCL，为后续I2C通信准备空闲状态。

*/

```
}
```

/**

* @brief I2C开始

*/

```
void OLED_I2C_Start(void)
```

```
{
```

```
    OLED_W_SDA(1);
```

```
    OLED_W_SCL(1);
```

```
    OLED_W_SDA(0);
```

/*

I2C协议：起始信号 = SCL高时，SDA从高到低跳变。

停止信号 = SCL高时，SDA从低到高跳变。

*/

```
    OLED_W_SCL(0); // 拉低SCL，便于后续传输数据时不误判起始/停止。
```

```
}
```

/**

* @brief I2C停止

*/

```
void OLED_I2C_Stop(void)
```

```
{
```

```
    OLED_W_SDA(0);
```

```
    OLED_W_SCL(1);
```

```
    OLED_W_SDA(1);
```

```
}
```

```

/**
 * @brief I2C发送一个字节
 * @param Byte 要发送的一个字节
 */
void OLED_I2C_SendByte(uint8_t Byte)
{
    uint8_t i;
    for (i = 0; i < 8; i++)
    {
        /* 问题：详细解释括号内，看不明白
        答：Byte & (0x80 >> i) 用于逐位取出Byte的最高位到最低位。
        i=0时 0x80>>0=0x80 (1000 0000)，Byte & 0x80 取出最高位；
        i=1时 0x80>>1=0x40 (0100 0000)，取出次高位.....依此类推。
        结果非0则写1，否则写0。这样依次发送8个数据位。 */
        OLED_W_SDA(Byte & (0x80 >> i));
        OLED_W_SCL(1); // 上升沿，从机采样
        OLED_W_SCL(0);
    }
    OLED_W_SCL(1); // 额外的一个时钟，不处理应答信号（忽略从机ACK）
    OLED_W_SCL(0);
    /* 问题：可以得到的结果，一般什么场景使用？
    答：此函数是I2C底层发送字节的通用函数，可用于任何I2C通信（写寄存器、写数据等）。
    补充：实际产品中最好检测应答位以确保通信可靠，这里简化处理。 */
}

/**
 * @brief OLED写命令
 */
void OLED_WriteCommand(uint8_t Command)
{
    OLED_I2C_Start();
    OLED_I2C_SendByte(0x78); // 从机地址
    /* 问题：这个是如何确定的？需要修改吗？
    答：0x78是OLED SSD1306的默认I2C从机地址（7位地址0x3C，左移一位加写位=0x78）。
    一般不需要改，除非硬件上SA0引脚接法不同（地址会变），参考OLED手册。 */
    OLED_I2C_SendByte(0x00); // 控制字节：后续为命令
    /* 问题：有什么作用？下一步呢？
    答：0x00表示后续发送的是命令字节（控制字节），OLED内部据此区分命令和数据。
    下一步发送真正的命令值（Command）。 */
    OLED_I2C_SendByte(Command);
    OLED_I2C_Stop();
}

/**
 * @brief OLED写数据，并更新缓冲区及当前列
 */
void OLED_WriteData(uint8_t Data)
{
    OLED_I2C_Start();
    OLED_I2C_SendByte(0x78); // 从机地址
    OLED_I2C_SendByte(0x40); // 控制字节：后续为数据
    OLED_I2C_SendByte(Data);
    OLED_I2C_Stop();
}

```

```

// 更新缓冲区并递增列
OLED_Buffer[current_page][current_col] = Data;
current_col++;
if (current_col >= 128) {
    current_col = 0; // 列地址回绕（实际上需根据OLED寻址模式，此处简化）
}
/* 问题：上一个函数和这个函数最大的区别就是有没有缓冲区，所以区别和意义是什么？使用场景呢？
    答：WriteData更新缓冲区，WriteCommand不更新。缓冲区用于维护屏幕内容的副本，便于后续画点、读回等。
    使用场景：需要单独修改某个像素或实现图形操作时，必须依赖缓冲区；纯文本显示可以不使用缓冲区，但画点功能就必须有。
    补充：此函数还自动维护当前列，但实际OLED内部列地址由硬件管理，这里是为了保持软件缓冲区与硬件同步。 */
}

/**
 * @brief OLED设置光标位置，并更新当前页和列
 */
void OLED_SetCursor(uint8_t Y, uint8_t X)
{
    current_page = Y;
    current_col = X;
    OLED_WriteCommand(0xB0 | Y); // 设置页地址 (0xB0~0xB7)
    OLED_WriteCommand(0x10 | ((X & 0xF0) >> 4)); // 设置列地址高4位 (0x10~0x1F)
    OLED_WriteCommand(0x00 | (X & 0x0F)); // 设置列地址低4位 (0x00~0x0F)
    /* 问题：这几行干了什么？写入了什么命令？
        答：OLED采用页寻址模式，先设置页（0~7），再设置列（0~127）。
        命令格式：0xB0+页号；列高四位通过0x10+高四位，低四位通过0x00+低四位。
        补充：列地址低4位命令通常写作0x00 | (X & 0x0F)，但实际SSD1306允许直接写0x00~0x0F，也可只写0x0F，标准做法是写0x00~0x0F。 */
}

/**
 * @brief OLED清屏，并清空缓冲区
 */
void OLED_Clear(void)
{
    uint8_t i, j;
    for (j = 0; j < 8; j++) {
        OLED_SetCursor(j, 0);
        for (i = 0; i < 128; i++) {
            OLED_WriteData(0x00); // 清空屏幕并更新缓冲区
        }
    }
}

/**
 * @brief OLED显示一个字符（8x16字体）
 */
void OLED_ShowChar(uint8_t Line, uint8_t Column, char Char)
{
    uint8_t i;
    OLED_SetCursor((Line - 1) * 2, (Column - 1) * 8); // 上半部分

```

```

/* 问题：这一步的意义是什么？如果不是不是就随机位置显示了？
   答：字符显示需要精确位置，否则会覆盖其他内容。这里将逻辑行（1~4）映射到实际页
   （0~7），列（1~16）映射到列（0~127）。
   如果不设光标，默认会在上次操作的位置继续，导致乱序。 */
for (i = 0; i < 8; i++)
{
    OLED_WriteData(OLED_F8x16[Char - ' '][i]);          // 上半部分数据
    /* 问题：Char - ' ' 是什么意思？
       答：字库数组下标从0开始，而ASCII空格' '是32，所以' '- ' '=0, 'A' - ' '=33, 对
       应字库中的索引。 */
}
OLED_SetCursor((Line - 1) * 2 + 1, (Column - 1) * 8);    // 下半部分
for (i = 0; i < 8; i++)
{
    OLED_WriteData(OLED_F8x16[Char - ' '][i + 8]);      // 下半部分数据
}
/* 问题：所以如果一个字符是8位，显示8位的16进制的数字，显示出一个字符，一个for循环中
   显示一个字符？那两个for循环的意义是什么？
   答：8x16字体，垂直方向16像素，分上下两页（每页8像素）。上半部分8字节对应上半8行，
   下半部分8字节对应下半8行。
   两个for循环分别写上半和下半，共同组成一个完整字符。 */
}

/**
 * @brief OLED显示字符串
 */
void OLED_ShowString(uint8_t Line, uint8_t Column, char *String)
{
    uint8_t i;
    for (i = 0; String[i] != '\0'; i++)
    {
        OLED_ShowChar(Line, Column + i, String[i]);
    }
}

/**
 * @brief OLED次方函数（内部使用）
 */
uint32_t OLED_Pow(uint32_t X, uint32_t Y)
{
    uint32_t Result = 1;
    while (Y--)
    {
        Result *= X;
    }
    return Result;
}
/* 问题：本质上就是一个数学运算，和OLED的实际应用有什么关联？
   答：用于显示数字时，计算各位的权重（10^(len-1-i)等），从而提取数字的每一位。 */
}

/**
 * @brief OLED显示数字（十进制，无符号）
 */
void OLED_ShowNum(uint8_t Line, uint8_t Column, uint32_t Number, uint8_t Length)

```

```

{
    uint8_t i;
    for (i = 0; i < Length; i++)
    {
        OLED_ShowChar(Line, Column + i, Number / OLED_Pow(10, Length - i - 1) % 10
+ '0');
        /* 问题：解释 Number / OLED_Pow(10, Length - i - 1) % 10 + '0' 是什么意思？
        答：先除以10^(剩余位数-1)得到最高位，再%10取个位，最后+'0'转为数字字符。
        例如 Number=123, Length=3, i=0: 123/100=1, 1%10=1, +'0'='1' */
    }
}

/**
 * @brief OLED显示数字（十进制，带符号）
 */
void OLED_ShowSignedNum(uint8_t Line, uint8_t Column, int32_t Number, uint8_t
Length)
{
    uint8_t i;
    uint32_t Number1;
    if (Number >= 0)
    {
        OLED_ShowChar(Line, Column, '+');
        Number1 = Number;
    }
    else
    {
        OLED_ShowChar(Line, Column, '-');
        Number1 = -Number;
    }
    for (i = 0; i < Length; i++)
    {
        OLED_ShowChar(Line, Column + i + 1, Number1 / OLED_Pow(10, Length - i - 1)
% 10 + '0');
    }
}

/**
 * @brief OLED显示数字（十六进制）
 */
void OLED_ShowHexNum(uint8_t Line, uint8_t Column, uint32_t Number, uint8_t
Length)
{
    uint8_t i, SingleNumber;
    for (i = 0; i < Length; i++)
    {
        SingleNumber = Number / OLED_Pow(16, Length - i - 1) % 16;
        /* 问题：显示16进制和10进制和2进制，OLED一般可以显示什么进制？几个不一样的进制，
        是不是只有这行的数字不一样就可以了？
        答：OLED本身只显示字符，这些函数本质是把数值转换成对应的ASCII字符。对于16进
        制，需要把0~15转为'0'-'9'和'A'-'F'。
        不同进制只需改变基数（10/16/2）和转换逻辑，其他都一样。 */
        if (SingleNumber < 10)
        {

```

```

        OLED_ShowChar(Line, Column + i, SingleNumber + '0');
        /* 问题：为什么加0？
           答：将数字0~9转为字符'0'~'9'，ASCII码差值为'0'的值。 */
    }
    else
    {
        OLED_ShowChar(Line, Column + i, SingleNumber - 10 + 'A');
        /* 问题：为什么-10后加A？
           答：将10~15转为'A'~'F'，ASCII码'A'对应65，10-10=0+'A'='A'。 */
    }
}

/**
 * @brief OLED显示数字（二进制）
 */
void OLED_ShowBinNum(uint8_t Line, uint8_t Column, uint32_t Number, uint8_t
Length)
{
    uint8_t i;
    for (i = 0; i < Length; i++)
    {
        OLED_ShowChar(Line, Column + i, Number / OLED_Pow(2, Length - i - 1) % 2 +
'0');
    }
}

/**
 * @brief OLED初始化
 */
void OLED_Init(void)
{
    uint32_t i, j;
    for (i = 0; i < 1000; i++)          // 上电延时
    {
        for (j = 0; j < 1000; j++);
    }
    /* 问题：这一步为什么需要有？
       答：OLED上电后需要等待内部复位稳定，否则后续命令可能无效。这是简单的延时，也可用精
       确的定时器。 */

    OLED_I2C_Init();                    // 端口初始化

    /* 以下为SSD1306标准初始化命令序列 */
    OLED_WriteCommand(0xAE);            // 关闭显示

    OLED_WriteCommand(0xD5);            // 设置显示时钟分频比/振荡器频率
    OLED_WriteCommand(0x80);

    OLED_WriteCommand(0xA8);            // 设置多路复用率（64行）
    OLED_WriteCommand(0x3F);

    OLED_WriteCommand(0xD3);            // 设置显示偏移
    OLED_WriteCommand(0x00);

```

```

    OLED_WriteCommand(0x40);    // 设置显示开始行 (0行)

    OLED_WriteCommand(0xA1);    // 设置段重映射 (0xA1正常, 0xA0左右反置)

    OLED_WriteCommand(0xC8);    // 设置COM扫描方向 (0xC8正常, 0xC0上下反置)

    OLED_WriteCommand(0xDA);    // 设置COM引脚硬件配置
    OLED_WriteCommand(0x12);

    OLED_WriteCommand(0x81);    // 设置对比度控制
    OLED_WriteCommand(0xCF);

    OLED_WriteCommand(0xD9);    // 设置预充电周期
    OLED_WriteCommand(0xF1);

    OLED_WriteCommand(0xDB);    // 设置VCOMH取消选择级别
    OLED_WriteCommand(0x30);

    OLED_WriteCommand(0xA4);    // 设置整个显示打开/关闭 (0xA4恢复显示, 0xA5全亮)

    OLED_WriteCommand(0xA6);    // 设置正常/倒转显示 (0xA6正常, 0xA7反显)

    OLED_WriteCommand(0x8D);    // 设置充电泵 (必须开启, 否则屏幕不亮)
    OLED_WriteCommand(0x14);

    OLED_WriteCommand(0xAF);    // 开启显示

    OLED_Clear();               // OLED清屏

    /* 问题: 这些里面什么需要掌握和未来嵌入式研发迁移需要修改的? 重要的?
       答: 这些初始化序列基本固定, 不同驱动IC略有差异。迁移时需根据新屏幕的数据手册修改命令值。
       关键是理解每一组命令的含义 (如对比度、复用率、电荷泵等), 但大多无需改动, 直接复制即可。
       补充: 0x8D 0x14 是电荷泵使能, 非常重要, 忘记开启屏幕无显示。 */
}

/**
 * @brief OLED显示汉字 (支持正反显示)
 */
void OLED_ShowCN(uint8_t Line, uint8_t Column, uint8_t Num, uint8_t mode)
{
    uint8_t i;
    uint8_t page_start = (Line - 1) * 2;    // 计算起始页 (汉字占2页)
    uint8_t col_start = (Column - 1) * 16;  // 计算起始列 (汉字占16列)

    // 显示上半部分 (前16字节)
    OLED_SetCursor(page_start, col_start);
    for (i = 0; i < 16; i++)
    {
        uint8_t data = OLED_F10x16[Num][i];
        OLED_WriteData(mode ? data : ~data); // 根据mode取反 (反显)
    }
}

```

```

// 显示下半部分 (后16字节)
OLED_SetCursor(page_start + 1, col_start);
for (i = 16; i < 32; i++)
{
    uint8_t data = OLED_F10x16[Num][i];
    OLED_WriteData(mode ? data : ~data);
}
}

/**
 * @brief OLED画点函数
 */
void OLED_DrawPoint(uint8_t x, uint8_t y, uint8_t mode)
{
    if (x >= 128 || y >= 64) return; // 坐标越界处理

    uint8_t page = y / 8; // 计算页地址
    uint8_t bit_pos = y % 8; // 计算页内位位置 (0~7)

    // 修改缓冲区对应位
    if (mode) {
        OLED_Buffer[page][x] |= (1 << bit_pos); // 点亮像素
    } else {
        OLED_Buffer[page][x] &= ~(1 << bit_pos); // 熄灭像素
    }

    // 更新到OLED屏幕
    OLED_SetCursor(page, x);
    OLED_WriteData(OLED_Buffer[page][x]);
    /* 补充：画点函数是图形操作的基础，必须依赖缓冲区，因为需要读取原有数据再修改某一位。
       如果不使用缓冲区，则无法实现“读-改-写”操作，只能整页覆盖。 */
}

/**
 * @brief OLED显示位图 (支持任意位置和尺寸)
 */
void OLED_ShowBMP(uint8_t x, uint8_t y, uint8_t width, uint8_t height, const
uint8_t *bmp, uint8_t mode)
{
    uint8_t page_end = y + height / 8;
    uint8_t col_end = x + width;
    uint16_t index = 0;

    for (uint8_t page = y; page < page_end; page++)
    {
        OLED_SetCursor(page, x);
        for (uint8_t col = x; col < col_end; col++)
        {
            uint8_t data = bmp[index++];
            OLED_WriteData(mode ? data : ~data); // 根据mode取反
        }
    }
    /* 补充：位图数据要求宽度和高度必须是8的倍数，因为页寻址最小单位是8行。此函数可显示任

```

```

意图片。 */
}

/* 总结补充：
1. 缓冲区是绘图的核心，所有修改屏幕的函数（画点、清屏等）都应同步缓冲区。
2. 初始化命令序列中电荷泵（0x8D 0x14）最易被忽略，导致屏幕不亮。
3. I2C通信中忽略应答位虽简化，但在噪声大的环境下可能出错。
4. OLED显示字符时，实际使用字模取模方式，需与取模软件格式匹配（通常为列行式、逆向、阴码）。
5. 此代码使用软件模拟I2C，移植到其他MCU时只需修改GPIO操作宏即可。
*/

```

【附件3】项目包4

PWM驱动电机的函数 Servo基本都是PWM的函数进行一定的线性变化使得可以直接适配于180度的舵机，所以重点是PWM的函数

```

#include "stm32f10x.h" // Device header

void PWM_Init(void)
{
    // 问题：为什么是APB1？为什么是TIM2？其他可以吗？
    // 答：TIM2-TIM7挂在APB1总线上；TIM2是通用定时器，有4个PWM通道。可以换TIM3/4/5，但要改时钟和引脚。
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM2, ENABLE);

    // 问题：初始化GPIO，都是使用APB2吗？后面可以写GPIOA也可以写GPIOB是吗？
    // 答：所有GPIO时钟都在APB2上。可以写GPIOB，但必须保证该引脚是当前定时器通道的复用功能引脚（查手册）。
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);

    GPIO_InitTypeDef GPIO_InitStructure;
    // 问题：这个为什么？这和点亮LED是一样的吧？
    // 答：不一样。LED用普通推挽(Out_PP)，CPU控制；PWM必须用复用推挽(AF_PP)，定时器硬件自动控制。
    // 归类：LED、蜂鸣器→Out_PP；PWM、SPI→AF_PP；按键→输入；模拟传感器→AIN；I2C→AF_OD。
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;

    // 问题：对于选择的口子有什么要求吗？比如不能占用什么口？必须GPIOB之类的？
    // 答：必须选该定时器通道的专用复用引脚（数据手册）。例如TIM2_CH2默认是PA1，重映射后可PB3。
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_1;

    // 问题：这一参数不变化了吗？
    // 答：可变化，可选2/10/50MHz。原则：高频PWM用50MHz，低速信号用2MHz降低干扰。
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    // 问题：必须是TIM2吗？下面的参数分别什么意思，可以变化吗？
    // 答：不必须，可用其他定时器。参数含义和可变性见下方注释。

```

```

TIM_InternalClockConfig(TIM2);

TIM_TimeBaseInitTypeDef TIM_TimeBaseInitStructure;
TIM_TimeBaseInitStructure.TIM_ClockDivision = TIM_CKD_DIV1;    // 时钟分频（滤波
用），通常DIV1，可改
TIM_TimeBaseInitStructure.TIM_CounterMode = TIM_CounterMode_Up; // 计数模式（向
上/中央对齐），可改
TIM_TimeBaseInitStructure.TIM_Period = 20000 - 1;    // ARR, 决定周期，必须按需求
计算
TIM_TimeBaseInitStructure.TIM_Prescaler = 72 - 1;    // PSC, 决定计数频率，必须按
需求计算
TIM_TimeBaseInitStructure.TIM_RepetitionCounter = 0; // 高级定时器专用，通用定时
器无效，写0
TIM_TimeBaseInit(TIM2, &TIM_TimeBaseInitStructure);

TIM_OCInitTypeDef TIM_OCInitStructure;
TIM_OCStructInit(&TIM_OCInitStructure);
TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM1;
TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_High; // 高极性（有效电平
为高），可改
TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable; // 输出使能
TIM_OCInitStructure.TIM_Pulse = 0; // CCR初始值，运行时通过函数修改
TIM_OC2Init(TIM2, &TIM_OCInitStructure); // 初始化通道2，对应PA1（TIM2_CH2）

// 问题：什么作用？没有会怎么样？什么时候不需要？
// 答：启动定时器计数器。没有则CNT永远为0，无PWM波形（固定电平）。需要暂停输出时可设
为DISABLE。
TIM_Cmd(TIM2, ENABLE);
}

// 问题：这是最常用的函数吗？还有什么很重要的PWM相关的函数吗？这一函数什么用？Compare1-4
的区别是什么？
// 答：是，运行时改占空比最常用。其他重要：TIM_SetAutoreload()改频率，TIM_Cmd()启停。
// 该函数修改CCR2的值（占空比）。Compare1-4对应四个独立通道，各控制不同引脚，输出不同
占空比（共用频率）。
void PWM_SetCompare2(uint16_t Compare)
{
    TIM_SetCompare2(TIM2, Compare);
}

```